

目录

实验六：进程管理与调度实验报告	2
一、实验基本信息	2
（一）实验目的	2
（二）实验环境	2
二、实验核心原理	2
（一）进程结构体与状态定义	2
（二）时间片轮转调度算法	2
（三）进程同步机制	2
三、实验步骤与关键代码	2
（一）任务 1：进程核心函数实现	2
（二）任务 2：进程调度机制实现	5
（三）任务 3：sleep/wakeup 同步原语实现	8
（四）任务 4：首个进程创建优化	9
（五）运行结果	9
四、实验关键问题解答	11
（一）进程管理核心问题	11
（二）调度机制问题	11
（三）同步机制问题	12
（四）性能与扩展性问题	12

实验六：进程管理与调度实验报告

[仓库地址在此处](#)

此阶段代码请通过 `git checkout process` 来查看。

一、实验基本信息

（一）实验目的

1. 完善进程结构体定义与核心系统调用（fork、exit、wait 等），实现进程完整生命周期管理。
2. 基于时间片轮转算法实现进程调度机制，支持多进程上下文切换与 CPU 资源分配。
3. 实现 sleep/wakeup 进程同步原语，解决进程等待与唤醒的协调问题，避免丢失唤醒等异常。
4. 深入理解进程状态转换逻辑、调度时机及多核环境下的进程管理优化思路。

（二）实验环境

- 操作系统：xv6
- 架构：RISC-V
- 开发工具：GCC、QEMU（模拟器）、GDB（调试）、汇编器

二、实验核心原理

（一）进程结构体与状态定义

1. 进程结构体扩展：在原有基础上新增自旋锁、父进程指针、退出状态、睡眠等待对象等字段，支撑多进程管理与同步。
2. 进程状态枚举：定义 UNUSED（未使用）、RUNNABLE（就绪）、RUNNING（运行）、SLEEPING（睡眠）、ZOMBIE（僵尸）五种状态，覆盖完整生命周期。

（二）时间片轮转调度算法

核心逻辑：为每个进程分配固定时间片，时钟中断触发时递减当前进程时间片，时间片为 0 时触发调度，切换至下一个就绪进程，同时重置当前进程时间片，保证进程公平占用 CPU。

（三）进程同步机制

sleep/wakeup 机制：进程等待资源时调用 sleep 进入睡眠状态并释放 CPU，资源就绪时通过 wakeup 唤醒对应进程，切换为就绪态等待调度，避免忙等待导致的 CPU 浪费。

三、实验步骤与关键代码

（一）任务 1：进程核心函数实现

1. 进程初始化与资源管理

```
1 // 全局变量定义
2 static proc_t procs[NPROC];
```



```
3  static proc_t* proczero;
4  static int global_pid = 1;
5  static spinlock_t lk_pid;
6
7  // 进程初始化函数
8  void proc_init(void) {
9      initlock(&lk_pid, "global_pid");
10     // 初始化进程数组, 设置内核栈
11     for (int i = 0; i < NPROC; i++) {
12         procs[i].state = UNUSED;
13         procs[i].kstack = KSTACK((uint64)&procs[i]);
14         proc_free(&procs[i]);
15     }
16     // 创建首个进程proczero
17     proczero = proc_alloc();
18     proczero->pid = 0;
19     proczero->state = RUNNABLE;
20 }
21
22 // 进程分配函数
23 proc_t* proc_alloc(void) {
24     proc_t *p;
25     acquire(&lk_pid);
26     // 查找未使用的进程槽位
27     for (p = procs; p < &procs[NPROC]; p++) {
28         if (p->state == UNUSED) {
29             // 初始化进程字段
30             p->state = USED;
31             p->pid = global_pid++;
32             initlock(&p->lk, "proc");
33             // 分配页表与内核栈
34             p->pgtbl = proc_pgtbl_init((uint64)p->tf);
35             p->tf = (trapframe_t*)pmem_alloc(PGSIZE);
36             memset(p->tf, 0, PGSIZE);
37             release(&lk_pid);
38             return p;
39         }
40     }
41     release(&lk_pid);
42     return NULL; // 进程表满
43 }
44
45 // 进程资源释放函数
46 void proc_free(proc_t *p) {
47     acquire(&p->lk);
48     // 释放页表、陷阱帧等资源
49     pgtbl_free(p->pgtbl);
```

```
50     pmem_free((uint64)p->tf, PGSIZE);
51     // 重置进程状态与字段
52     p->state = UNUSED;
53     p->parent = NULL;
54     p->exit_state = 0;
55     p->sleep_space = NULL;
56     release(&p->lk);
57 }
```

2. fork/exit/wait 系统调用实现

```
1  // fork系统调用: 创建子进程
2  int sys_fork(void) {
3      proc_t *parent = myproc();
4      proc_t *child = proc_alloc();
5      if (child == NULL) return -1;
6
7      // 设置父进程与子进程关系
8      child->parent = parent;
9      // 复制父进程用户内存空间与陷阱帧
10     copyproc(parent, child);
11     // 子进程返回值设为0, 父进程返回子进程PID
12     child->tf->a0 = 0;
13     // 复制打开的文件描述符
14     for (int i = 0; i < NOFILE; i++) {
15         child->ofile[i] = parent->ofile[i];
16     }
17     // 子进程设为就绪态
18     child->state = RUNNABLE;
19     return child->pid;
20 }
21
22 // exit系统调用: 进程退出
23 void sys_exit(int status) {
24     proc_t *p = myproc();
25     acquire(&p->lk);
26     // 设置退出状态, 切换为僵尸态
27     p->exit_state = status;
28     p->state = ZOMBIE;
29     // 托付子进程给proczero
30     proc_reparent(p);
31     // 唤醒父进程
32     proc_wakeup_one(p->parent);
33     release(&p->lk);
34     // 放弃CPU, 触发调度
35     proc_yield();
36 }
37
```

```
38 // wait系统调用: 等待子进程退出
39 int sys_wait(int *status) {
40     proc_t *parent = myproc();
41     while (1) {
42         int found = 0;
43         // 遍历查找子进程
44         for (proc_t *p = procs; p < &procs[NPROC]; p++) {
45             acquire(&p->lk);
46             if (p->parent == parent && p->state == ZOMBIE) {
47                 // 读取退出状态
48                 if (status != NULL) {
49                     *status = p->exit_state;
50                 }
51                 // 释放僵尸进程资源
52                 proc_free(p);
53                 found = 1;
54             }
55             release(&p->lk);
56             if (found) return 0;
57         }
58         // 未找到僵尸进程, 进入睡眠
59         proc_sleep(parent, &parent->lk);
60     }
61 }
62
63 // 进程托付函数: 父进程退出时托付子进程给proczero
64 void proc_reparent(proc_t *p) {
65     for (proc_t *child = procs; child < &procs[NPROC]; child++) {
66         if (child->parent == p) {
67             child->parent = proczero;
68         }
69     }
70 }
```

(二) 任务 2: 进程调度机制实现

1. 上下文切换函数

```
1 // 上下文结构体定义
2 struct context {
3     uint64 ra; // 返回地址
4     uint64 sp; // 栈指针
5     // 被调用者保存寄存器, 上下文切换需保存
6     uint64 s0;
7     uint64 s1;
8     uint64 s2;
9     uint64 s3;
10    uint64 s4;
```



```
11     uint64 s5;
12     uint64 s6;
13     uint64 s7;
14     uint64 s8;
15     uint64 s9;
16     uint64 s10;
17     uint64 s11;
18 };
19
20 // 汇编实现上下文切换 (swtch.S)
21 .globl swtch
22 swtch:
23     # 保存旧上下文 (被调用者保存寄存器)
24     sd s0, 0(a0)
25     sd s1, 8(a0)
26     sd s2, 16(a0)
27     sd s3, 24(a0)
28     sd s4, 32(a0)
29     sd s5, 40(a0)
30     sd s6, 48(a0)
31     sd s7, 56(a0)
32     sd s8, 64(a0)
33     sd s9, 72(a0)
34     sd s10, 80(a0)
35     sd s11, 88(a0)
36     sd ra, 96(a0)
37     sd sp, 104(a0)
38
39     # 恢复新上下文
40     ld s0, 0(a1)
41     ld s1, 8(a1)
42     ld s2, 16(a1)
43     ld s3, 24(a1)
44     ld s4, 32(a1)
45     ld s5, 40(a1)
46     ld s6, 48(a1)
47     ld s7, 56(a1)
48     ld s8, 64(a1)
49     ld s9, 72(a1)
50     ld s10, 80(a1)
51     ld s11, 88(a1)
52     ld ra, 96(a1)
53     ld sp, 104(a1)
54
55     ret
```

2. 调度器核心函数

```
1  #define TIME_SLICE 10 // 时间片大小（时钟中断次数）
2
3  // 调度触发函数：保存上下文并切换到调度器
4  void proc_sched(void) {
5      proc_t *p = myproc();
6      cpu_t *c = mycpu();
7      // 检查锁状态与中断状态
8      if (holding(&p->lk) || !intr_get()) {
9          panic("proc_sched: invalid state");
10     }
11     // 保存当前进程状态为就绪态
12     p->state = RUNNABLE;
13     // 切换到CPU调度器上下文
14     swtch(&p->ctx, &c->ctx);
15     // 恢复中断状态
16     intr_on();
17 }
18
19 // 调度器主函数：时间片轮转选择就绪进程
20 void proc_scheduler(void) {
21     cpu_t *c = mycpu();
22     c->proc = NULL;
23     proc_t *p;
24     for (;;) {
25         intr_on(); // 开启中断，允许时钟中断触发调度
26         for (p = procs; p < &procs[NPROC]; p++) {
27             acquire(&p->lk);
28             if (p->state == RUNNABLE) {
29                 // 切换进程状态为运行态
30                 p->state = RUNNING;
31                 c->proc = p;
32                 // 重置时间片
33                 p->time_slice = TIME_SLICE;
34                 // 切换上下文到目标进程
35                 swtch(&c->ctx, &p->ctx);
36                 // 调度返回后重置当前进程
37                 c->proc = NULL;
38             }
39             release(&p->lk);
40         }
41     }
42 }
43
44 // 进程放弃CPU：主动触发调度
45 void proc_yield(void) {
46     acquire(&myproc()->lk);
```

```
47     proc_sched();
48     release(&myproc()->lk);
49 }
50
51 // 时钟中断处理：递减时间片，触发调度
52 void timer_handler(void) {
53     proc_t *p = myproc();
54     if (p == NULL) return;
55     // 时间片递减，为0时触发调度
56     if (--p->time_slice <= 0) {
57         proc_yield();
58     }
59     // 唤醒超时进程
60     timer_update();
61 }
```

(三) 任务 3: sleep/wakeup 同步原语实现

```
1 // 进程睡眠函数：等待指定对象 
2 void proc_sleep(void *chan, spinlock_t *lk) {
3     proc_t *p = myproc();
4     // 释放传入的锁，避免死锁
5     release(lk);
6     acquire(&p->lk);
7     // 设置睡眠对象，切换状态为睡眠态
8     p->sleep_space = chan;
9     p->state = SLEEPING;
10    // 触发调度，释放CPU
11    proc_sched();
12    // 唤醒后重新获取锁
13    acquire(lk);
14    release(&p->lk);
15 }
16
17 // 唤醒单个进程：仅唤醒指定父进程
18 void proc_wakeup_one(proc_t *parent) {
19     acquire(&parent->lk);
20     if (parent->state == SLEEPING) {
21         parent->state = RUNNABLE;
22     }
23     release(&parent->lk);
24 }
25
26 // 唤醒所有等待对象的进程
27 void proc_wakeup(void *chan) {
28     for (proc_t *p = procs; p < &procs[NPROC]; p++) {
29         acquire(&p->lk);
30         if (p->state == SLEEPING && p->sleep_space == chan) {
```

```
31         p->state = RUNNABLE;
32         p->sleep_space = NULL;
33     }
34     release(&p->lk);
35 }
36 }
```

(四) 任务 4：首个进程创建优化

```
1 void proc_make_first(void) {
2     // 直接调用proc_alloc分配首个进程
3     proczero = proc_alloc();
4     proczero->pid = 0;
5     proczero->parent = proczero; // 自身为父进程
6     // 初始化用户页表与陷阱帧
7     proczero->pgtbl = proc_pgtbl_init((uint64)proczero->tf);
8     proczero->tf->epc = USER_CODE;
9     proczero->tf->sp = USER_STACK;
10    // 设置进程状态为就绪态，无需直接切换
11    proczero->state = RUNNABLE;
12 }
```

(五) 运行结果

运行下面的 initcode.c，执行结果如图 1 所示：

```
1 #include "user.h"
2 #include "memlayout.h"
3
4 void test_fork();
5
6 void test_sbrk();
7 int main(){
8     printf("Hello, World!\n");
9
10    test_sbrk();
11
12    test_fork();
13
14    shutdown();
15    while(1);
16    return 0;
17 }
18
19 void test_fork(){
20
21     int pid = fork();
22     int status;
23     if (pid < 0) {
```

```
24     printf("Fork failed!\n");
25 } else if (pid == 0) {
26     // Child process
27     printf("Hello from child process! PID: %d\n", getpid());
28     exit(0);
29 } else {
30     // Parent process
31     wait(&status);
32     printf("Hello from parent process! PID: %d, Child PID: %d\n",
33           getpid(), pid);
34     printf("Child exited with status: %d\n", status);
35 }
36 }
37
38 void test_sbrk(){
39
40
41     long long addr = sbrk(0);
42
43     printf("Current break address: %p\n", addr);
44
45     sbrk(addr + 4096); // 增加4KB
46
47     // 测试是否成功增加
48     // 写入 'test sbrk'
49
50     char *test_str = "test sbrk";
51     int n = 9;
52     for(int i = 0; i < n; i++) {
53         *((char*)addr + i) = test_str[i];
54     }
55     *((char*)addr + n) = '\0';
56     printf("%s\n", (char*)addr);
57 }
58
59 }
```

```
~~

xv6 is booting!
proc 1: pagetable at 0x0000000087f75000
proc 1: trapframe at 0x0000000087f76000
proc 1: kernel stack at 0x0000003fffffd000
proc 1: context at 0x0000003fffffe000
sz: 1888
prog_pages: 1, total_pages: 2, total_size: 8192
Hello, World!
Current break address: 0x0000000000002000
test sbrk
proc 2: pagetable at 0x0000000087f6a000
proc 2: trapframe at 0x0000000087f6b000
proc 2: kernel stack at 0x0000003fffffb000
proc 2: context at 0x0000003fffffc000
Hello from parent process! PID: 1, Child PID: 2
Child exited with status: 0
```

图 1: initcode 运行结果

四、实验关键问题解答

(一) 进程管理核心问题

1. 父进程退出子进程未退出的处理: 通过 `proc_reparent` 函数将子进程托付给根进程 `proczero`, 由于 `proczero` 作为进程树根部不会退出, 确保子进程始终有父进程可调用 `wait` 回收资源, 避免孤儿进程资源泄漏。
2. `wait` 函数避免忙等待的方案: 将原循环中的 `proc_yield` 替换为 `proc_sleep`, 父进程未找到僵尸子进程时进入睡眠态, 释放 CPU 给其他进程; 子进程退出时通过 `proc_wakeup_one` 主动唤醒父进程, 大幅降低 CPU 占用。
3. 进程状态转换触发条件:
 - `UNUSED`→`USED`: `proc_alloc` 分配进程槽位时。
 - `USED`→`RUNNABLE`: 进程初始化完成、`sleep` 唤醒、时间片到调度时。
 - `RUNNABLE`→`RUNNING`: 调度器选中进程并切换上下文时。
 - `RUNNING`→`SLEEPING`: 调用 `proc_sleep` 等待资源时。
 - `RUNNING`→`ZOMBIE`: 调用 `sys_exit` 退出时。
 - `ZOMBIE`→`UNUSED`: 父进程调用 `wait` 回收资源时。

(二) 调度机制问题

1. 时钟中断触发调度的原因: 时钟中断是系统周期性触发的事件, 可确保每个进程占用 CPU 的时间不超过设定的时间片, 实现公平的时间片轮转调度, 避免单个进程长期占用 CPU。
2. 轮转调度的公平性: 轮转调度为每个进程分配相同的时间片, 按顺序轮流执行, 确保所有就绪进程获得平等的 CPU 使用权, 无优先级差异时公平性较高, 但未考虑进程优先级与紧急程度, 实时性较差。
3. 上下文切换保存寄存器的选择: 仅保存被调用者保存寄存器 (`s0-s11`、`ra`、`sp`), 原因是调用者保存寄存器由函数调用者负责保存, 上下文切换过程中无需内核额外

保存，可减少切换开销；被调用者保存寄存器需由被调用者保存，内核切换时必须主动保存以保证进程恢复后执行一致性。

（三）同步机制问题

1. **sleep/wakeup 避免丢失唤醒的锁规则：**sleep 前释放用户传入的锁，唤醒后重新获取；wakeup 操作需持有被唤醒进程的锁，确保进程睡眠状态的修改与唤醒操作原子性，避免进程在睡眠状态切换过程中丢失唤醒信号。
2. **自旋锁与 sleep 的区别：**自旋锁获取不到资源时 CPU 空转等待，适用于短时间等待场景；sleep 获取不到资源时进程进入睡眠态并释放 CPU，适用于长时间等待场景，可提高 CPU 利用率。

（四）性能与扩展性问题

1. **fork 性能瓶颈解决：**fork 的核心瓶颈是复制父进程内存空间，可采用写时复制（Copy-On-Write）优化，初始时父子进程共享物理内存，仅当某一方修改内存时才复制对应页面，减少内存拷贝开销。
2. **上下文切换开销降低方案：**减少上下文切换时保存的寄存器数量（仅保留必要的被调用者保存寄存器）、优化 swtch 汇编指令执行效率、合并频繁的调度操作，降低切换频率。
3. **多核调度与负载均衡：**多核环境下为每个 CPU 配备独立调度器，维护本地就绪进程队列；定期检查各 CPU 队列的进程数量，将进程从负载高的 CPU 迁移到负载低的 CPU，实现负载均衡。